nexOOS ETL Pipeline Architecture

Enterprise Analytical Data Flow & Medallion Pipeline Blueprint

This document maps the comprehensive ETL (Extract, Transform, Load) Pipeline architecture for **nexOOS System 4 (Online Ordering System)**. It serves as the master data engineering and enterprise architecture guide linking our distributed NestJS microservices and Next.js frontend directly to our governed Kafka data stream and AWS-based Analytics Data Lake.

🟦 1. SOURCE LAYER (Microservices Schema)

The source tier consists of **three distinct Supabase (PostgreSQL) instances** isolated at the system level to separate service ownership and secure operational boundaries:

A. Distinct Supabase Database Instances

1. **Primary OOS Database (Auth, Order, Settings & Locations)**
   - **Connection Environment Variables:** `OOS_AUTH_SUPABASE_URL` / `OOS_ORDER_SUPABASE_URL` / `OOS_DELIVERY_SUPABASE_URL`
   - **Target Schemas/Tables:**
     - `public.customers` – Detailed customer profiles, authentication metadata, and lockout states.
     - `public.staff` – Admin, manager, and fulfillment staff roles and onboarding records.
     - `public.refresh_token_families` – Security token rotation logs.
     - `public.browsing_history` – Individual client browsing event counts.
     - `public.search_analytics` – Flat-file and table search string registers.
     - `public.audit_logs` – Admin dashboard security and change audit trails.
     - `public.oos_settings` – Runtime configurations and thresholds.
     - `public.branches` – Geographic and scheduling parameters of brick-and-mortar branches.
     - `public.online_orders` – Master order transaction table.
     - `public.online_order_items` – Line items mapped to orders.
     - `public.return_requests` – Post-delivery refund and replacement tickets.

2. **POS / Inventory Supabase Database (Read-Only POS Bridge)**
   - **Connection Environment Variables:** `OOS_CATALOG_SECOND_SUPABASE_URL` in `catalog-service`
   - **Target Schemas/Tables:**
     - `public.products` – Master product SKU listing, descriptions, pricing, and classifications.
     - `public.storebranches` – Replicated POS store structures.
     - `public.branch_inventory` – Live stock availability (`quantity_on_hand` and `quantity_reserved`) per SKU per branch.

3. **Cart Supabase Database (Isolated State Store)**
   - **Connection Environment Variables:** `OOS_CART_SUPABASE_URL` in `cart-service`
   - **Target Schemas/Tables:**
     - `public.cart_items` – Uncommitted customer shopping carts (isolated to prevent high-write operational noise from entering transaction tables).

B. Channel Discriminator Audit

- **Shared Tables:** `public.transactions` and `public.receipts` are structurally shared with System 1 (POS).
- **Discriminator Implementation:**
  - Writes to the shared transaction log include a soft tag: `cashier_name = 'Ecommerce'` (`order.service.ts` line 333 & 473).
  - A relational foreign key is mapped via `public.online_orders.transaction_id` pointing to `transactions.id`.
- **Analytical Safety:** All downstream OOS analytics read exclusively from `public.online_orders` and `public.online_order_items`, which are channel-isolated and prevent contamination from POS-only store transactions.

🟥 2. EXTRACT LAYER

Data extraction from the source databases is handled using dedicated Supabase Clients and background execution scripts:

A. Node.js Supabase Clients

- **Client Framework:** `@supabase/supabase-js`
- **Service Instantiation:** Configured dynamically inside service wrapper files:
  - `greenovate-be/auth-service/src/auth/supabase.service.ts`
  - `greenovate-be/order-service/src/order/supabase.service.ts`
  - `greenovate-be/catalog-service/src/catalog/supabase.service.ts`
  - `greenovate-be/cart-service/src/cart/supabase.service.ts`
  - `greenovate-be/delivery-service/src/delivery/supabase.service.ts`

B. Extraction Methods & Functions

- **Real-time Extraction:**
  - `listCustomerOrders(userId)` / `search(orderNumber, status, limit)` – Pulls transaction files from `online_orders`.
  - `adminListAllOrders(status, search, limit, offset)` – Extracts paginated administrative views.
  - `adminGetStats()` – Parallel database extractions aggregating operational counts.
- **Bulk/Batch Extraction:**
  - **File:** `greenovate-be/scripts/kafka-backfill.ts`
  - **Extraction Mechanism:** Paginated queries utilizing Supabase range boundaries to load large historical volumes:

    ```
    db.from(table).select(select).range(from, from + PAGE_SIZE - 1)
    ```

    - `PAGE_SIZE` is throttled at **1000** records to prevent resource exhaustion, and parallel batch chunks are processed with a `BATCH_SIZE = 50` and `BATCH_DELAY = 150ms`.

🟧 3. STAGING LAYER

The staging layer acts as the initial landing area where operational data is temporarily stored and backed up in raw form before downstream analytical parsing:

A. Supabase Transactional Tables

Raw checkout states are initially loaded into:

- `public.online_orders` (Raw header level metadata)
- `public.online_order_items` (Raw line items containing unparsed JSON payloads)
- `public.return_requests` (Raw refund claims with unstructured `items` JSONB columns)

B. Flat-File Staging

- **Log Location:** `search-analytics.log` (in project root)
- **Written By:** `analytics.service.ts` (NestJS) + `auth-service`
- **Content:** Appends sequential raw JSON strings mapping user queries:

```
{"query": "amoxicillin", "source": "search_bar", "timestamp": "2026-05-23T10:07:47Z"}
```

🟩 4. TRANSFORMATION LAYER (The Analytics Engine)

Our NestJS microservices and Next.js frontend implement a robust, multi-stage data transformation pipeline:

A. Data Validation & Cleaning Engine

- **Framework Validation:** Incoming API payloads are parsed and sanitized in backend controllers using **class-validator** DTO (Data Transfer Object) decorators:
  - Enforced using global NestJS `ValidationPipe` frameworks in each service's startup configuration.
- **Type Conversion & Sanitization:**
  - Safe parameter conversions (e.g. `parseLimit()` parsing string numbers, dynamic `Number()` coercions on money fields, and `.trim()` cleaning).
  - Role validation and boundary isolation via `requireAdminToken()` verifying custom JWT claims (`decoded.isAdmin === true` and `decoded.staffRole`).

B. Feature Engineering Engine

Our system transforms raw inputs into high-level features:

- **Descriptive Analytics:**
  - **KPI Totals:** Today's Order Count, Total Gross Revenue, and Order Pipeline status aggregates computed inside `order.service.ts` using `COUNT` and `SUM` Postgres aggregations:

    ```
    // Inside adminGetStats()
    const todayRevenue = SUM(total) WHERE created_at >= TODAY AND
    fulfillment_status != 'Cancelled'
    ```

  - **Client-Side Category Sales:** Aggregated in Next.js (`admin/page.tsx` line 145) via `buildCategory()` to map revenue groupings: $$\text{CategoryRevenue} = \sum (\text{item.price} \times \text{item.quantity})$
  - **Trending Searches:** Frequency-based 7-day query analysis aggregated inside `analytics.service.ts` : $$\text{SearchRank} = \text{Count}(\text{query}) \text{ grouped by } \text{query.toLowerCase().trim()}$$
  - **A affinity profiling:** Browsing habits aggregated in `auth.controller.ts` via `increment_product_view()` to personalize customer product lists based on view weights.
- **Predictive Analytics:**

    - **Apriori Association Engine:** Located in `order.controller.ts` (lines 280–396) as an API route (`POST /api/orders/internal/co-purchases`) and replicated client-side in `/admin/page.tsx` (`buildMarketBasket()`). It mines transaction lists to predict subsequent items a customer is likely to purchase:
        - Adaptive support: $\text{minSupport} = \max(2 / N, 0.05)$
        - Fixed confidence thresholds: $\text{minConfidence} = 0.30$

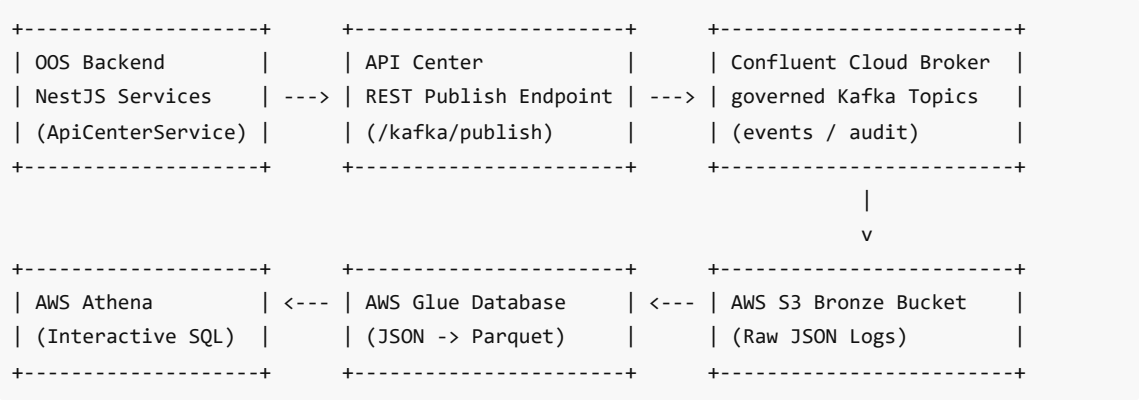
- **Prescriptive Analytics (Automated UI & Business Triggers):**
    - **Minimum Order Gate:** Blocks checkout dynamically in `Checkout.tsx` if `cartTotal < ₱100`.
    - **Free Shipping Trigger:** Sets delivery fee to ₱0 in `Checkout.tsx` if `subtotal >= ₱500`.
    - **Apriori Upsell Carousel:** Displays a "Customers also bought" recommendation tray at checkout, ranking items dynamically by mathematical `Lift` score.
    - **Fulfillment Alert Panel:** Displays an amber warning banner in the Admin Dashboard (`admin/page.tsx` line 534) if `pendingOrders + pendingReturns > 0` to prompt immediate dispatcher scheduling.

C. Modeling & Data Mining

- **Apriori Math Implementation:** Calculates transactional parameters:
    - $\text{Support}(A \cap B) = \frac{\text{Orders Containing Both A & B}}{\text{Total Orders } (N)}$
    - $\text{Confidence}(A \rightarrow B) = \frac{\text{Support}(A \cap B)}{\text{Support}(A)}$
    - $\text{Lift}(A \rightarrow B) = \frac{\text{Confidence}(A \rightarrow B)}{\text{Support}(B)}$ *(Values > 1 indicate positive associations)*
    - $\text{Leverage}(A \rightarrow B) = \text{Support}(A \cap B) - (\text{Support}(A) \times \text{Support}(B))$
    - $\text{Conviction}(A \rightarrow B) = \frac{1 - \text{Support}(B)}{\max(10^{-9}, 1 - \text{Confidence}(A \rightarrow B))}$
- **Fallback Systems:** If Apriori does not return recommendations, the system cascades to category fallback match logic.

🟪 5. KAFKA & AWS CLOUD STORAGE (Target/Load)

Transformed event files are published to our enterprise event bus and synchronized to long-term S3 storage:



A. Kafka Event Publishing Service

- **SDK Employed:** `@implementsprint/sdk` (GitHub Packages private auth flow).
- **Wrapper Implementation:** `ApiCenterService` inside NestJS microservices.
- **Publish Execution Method:**

```
async kafkaPublish(topic: string, eventType: string, payload: any, key?: string):
Promise<void>
```

- **Topic Routing Scheme:**
  - Domain events (`order_placed` , `fulfillment_status_changed` , `return_request_created` , `customer_registered` , `product_viewed`) are routed to:
    - `tribe.greenovate.events`
  - System and administrative audits (`admin_action`) are routed to:
    - `tribe.greenovate.audit`

B. S3 Bronze Data Lake & Medallion Pipeline

- **Storage Location:** `s3://nexoos-bronze/tribe=greenovate/topic={events, audit}/`
- **Trigger Constraint:** The Confluent Kafka S3 Connector flushes accumulated events in batches to S3 whenever the queue crosses **1000 events** (divided into ~250 event shares across POS, OOS, Supply Chain, and Rewards tribes).
- **Medallion Downstream:**
  - **Bronze Tier:** Raw, immutable JSON files.
  - **Silver Tier (AWS Glue):** Glue crawlers read the S3 Bronze folder and parse JSON structures into columnar **Parquet format** in a cataloged database.
  - **Gold Tier (AWS Athena):** Athena reads Silver schemas to compile analytical views for Power BI dashboards.

🟥 6. CONSUMPTION LAYER

The final tier of our architecture maps ingestion into user-facing platforms and corporate reporting engines:

A. Frontend Screens Consuming the APIs

There is **no React Native + Expo directory in this repository**. The mobile experience and administrative panel are built directly inside the responsive Next.js storefront **`nexOOS`** :

- `/src/app/checkout/Checkout.tsx` – Orchestrates order placement, promo validation, PayMongo payment session creations, and checkout recommendation lookups.
- `/src/app/account/Account.tsx` – Customer profile interface retrieving order and return histories.
- `/src/app/admin/page.tsx` – The main administrative dashboard rendering operational visual metrics (time series, fulfillment pies, categories, top products, and market basket grids).

B. Power BI CSV Schemas Required for Export

To ingest this system's data into corporate dashboards, the following schemas are exposed via Athena:

1. **`orders_export.csv`**

```
id (UUID) | order_number (VARCHAR) | receipt_number (VARCHAR) | customer_id (UUID) |
branch_id (UUID) | subtotal (DECIMAL) | delivery_fee (DECIMAL) | discount_amount
(DECIMAL) | total (DECIMAL) | promo_code (VARCHAR) | payment_method (VARCHAR) |
payment_status (VARCHAR) | fulfillment_status (VARCHAR) | created_at (TIMESTAMP)
```

2. **`order_items_export.csv`**

```
id (UUID) | online_order_id (UUID) | product_id (VARCHAR) | product_name (VARCHAR) |
category (VARCHAR) | unit_price (DECIMAL) | quantity (INTEGER) | line_total (DECIMAL)
```

3. `returns_export.csv`

```
id (UUID) | online_order_id (UUID) | customer_id (UUID) | receipt_number (VARCHAR) |
reason (VARCHAR) | status (VARCHAR) | created_at (TIMESTAMP)
```

4. `mba_rules_export.csv`

```
antecedent_product_id (VARCHAR) | consequent_product_id (VARCHAR) | support (DECIMAL)
| confidence (DECIMAL) | lift (DECIMAL) | leverage (DECIMAL) | conviction (DECIMAL)
```

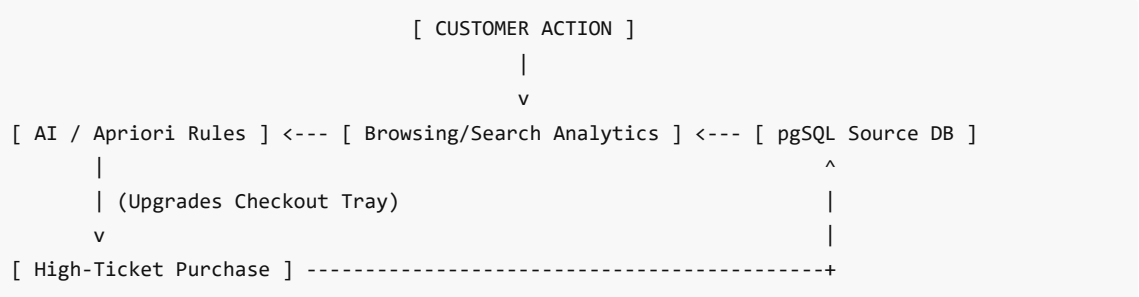
C. The Dashboard North Star Metric

> **"*Gross Online Revenue & Order Fulfillment Velocity Ratio*"**
>
> ***Core Business Question:*** *"How can we maximize gross online ordering revenue, accelerate the checkout conversion funnel, and optimize supply-chain procurement strategies by utilizing real-time product association rules, category demand surges, and fulfillment queue velocity metrics?"*

---

7. MODEL FEEDBACK LOOP

The system contains closed-loop patterns where analytical recommendations and admin interactions dynamically feedback into the source databases to modify future model weights:



1. **The Browsing Feedback Loop:**
   - **Action:** When a customer views a product, Next.js calls `POST /api/auth/product-view`.
   - **Write-back:** The auth service triggers the Supabase RPC function `increment_product_view(p_customer_id, p_product_id, p_category)`.
   - **Weight Adjustment:** This upserts `public.browsing_history` and increments `view_count`. Next time the customer checks out or searches, their category preference scores (`customer.browsing_history`) are recalculating, shifting the recommendation profiles served by the generative AI endpoints.
2. **The Search Feedback Loop:**
   - **Action:** Customers execute queries inside `/api/products/search`.
   - **Write-back:** Queries write immediately to `public.search_analytics` and append to `search-analytics.log`.
   - **Weight Adjustment:** Downstream indexing routines read top queries to automatically pin trending keywords on catalog pages and adjust catalog stock thresholds based on high search volumes.

3. **The Returns Feedback Loop:**
    - **Action:** Customers submit return/refund claims at `/api/orders/return-request`.
    - **Write-back:** Submits to `public.return_requests` with status `'pending'`.
    - **Weight Adjustment:** Admin decisions (`PATCH /orders/admin/returns/:id`) recalculate gross revenue net of returns, triggering immediate supply chain procurement alerts if SKU return ratios exceed configured margins.
4. **The Fulfillment Queue Feedback Loop:**
    - **Action:** Dispatchers adjust fulfillment states (`PATCH /orders/admin/status`).
    - **Write-back:** Updates `public.online_orders.fulfillment_status` to `'Delivered'` or `'Cancelled'`.
    - **Weight Adjustment:** Triggers real-time recalculations of dynamic delivery fee brackets and updates the historical fulfillment velocity matrix to adjust customer delivery time estimates.